UNIVERZITET U NIŠU
ELEKTRONSKI
FAKULTET

Obrada transakcija, planovi izvršavanja, izolacija i zaključavanje u MongoDB bazi podataka

Seminarski rad

Studijski program: Računarstvo i informatika
Modul: Softversko inženjerstvo

Student:
Anastasija Savov
Broj indeksa: 1924

Mentor:
Prof. dr Aleksandar Stanimirović

Niš, 2025.

Sadržaj

1 Uvod **3**

2 Model podataka i atomičnost operacija **4**
2.1 Dokumentni model i jednodokumentna atomičnost 4
2.2 Modelovanje podataka za smanjenje potrebe za transakcijama 5

3 Višedokumentne ACID transakcije **7**
3.1 ACID svojstva u MongoDB-u 7
3.2 Pokretanje i upravljanje transakcijama 7
3.3 Callback API za transakcije 7
3.4 Ograničenja višedokumentnih transakcija 8
3.5 Savepoints i načini pristupa transakcijama 9
3.6 Distribuirane transakcije na sharded klasterima 9

4 Planovi izvršavanja upita **11**
4.1 Query Optimizer u MongoDB-u 11
4.2 Metoda explain() 11
4.3 Ključne faze plana izvršavanja 12
4.4 Analiza efikasnosti upita 13
4.5 Plan Cache 14
4.6 Planovi agregacionog pipeline-a 15

5 Indeksi i optimizacija planova izvršavanja **17**
5.1 Tipovi indeksa u MongoDB-u 17
5.2 ESR pravilo za kompozitne indekse 19
5.3 Hint i forsiranje plana izvršavanja 20

6 Read Concern i Write Concern **21**
6.1 Write Concern 21
6.2 Read Concern 21
6.3 Kauzalna konzistentnost 22

7 Izolacija transakcija **23**
7.1 Snapshot izolacija 23
7.2 Write Conflict i optimistična kontrola konkurentnosti 23
7.3 Izgubljena ažuriranja (Lost Updates) 24
7.4 Serijalizabilnost i rasporedi izvršavanja 24

8 Zaključavanje u MongoDB-u **26**
8.1 WiredTiger Storage Engine i zaključavanje 26
8.2 Tipovi zaključavanja 26
8.3 Zaključavanje u praksi 26
8.4 Yielding - ustupanje lock-ova 27
8.5 Zaključavanje u višedokumentnim transakcijama 27
8.6 Deadlock i razrešavanje konflikata 27
8.7 MVCC Multiversion Concurrency Control 29
8.8 Veza sa Dvofaznim zaključavanjem (2PL) 29

8.9 Upravljanje vrućim tačkama (hot spots) 30

9 Preporuke za optimizaciju transakcija i konkurentnosti 31
9.1 Smanjenje trajanja transakcija 31
9.2 Izbor odgovarajućeg Read i Write Concern-a 31
9.3 Strategije za smanjenje write conflict-a 31
9.4 Praćenje performansi zaključavanja 32

10 Zaključak 34

11 Literatura 35

1 Uvod

MongoDB je jedna od najpopularnijih NoSQL baza podataka, zasnovana na dokumentnom modelu podataka u kome se podaci skladište u fleksibilnim BSON (Binary JSON) dokumentima. Za razliku od relacionih baza podataka koje koriste tabele sa fiksnom shemom, MongoDB omogućava skladištenje dokumenata različite strukture unutar iste kolekcije, što pruža veću fleksibilnost u modelovanju podataka [1].

Dugi niz godina, MongoDB je bio poznat po tome da ne podržava višedokumentne transakcije, oslanjajući se na atomičnost operacija nad pojedinačnim dokumentima i preporučujući modelovanje podataka koje minimizuje potrebu za transakcijama. Međutim, od verzije 4.0 (2018. godine) uvedena je podrška za višedokumentne ACID transakcije na replica set-ovima, a od verzije 4.2 i na sharded klasterima, čime je MongoDB dobio mogućnost koja ga stavlja u ravnopravan položaj sa relacionim bazama podataka u pogledu transakcionog integriteta [2].

Pored obrade transakcija, razumevanje planova izvršavanja upita, mehanizmi izolacije i strategija zaključavanja predstavljaju ključne aspekte optimizacije performansi MongoDB baze podataka. Planovi izvršavanja omogućavaju analizu načina na koji MongoDB obrađuje upite, dok nivoi izolacije i mehanizmi zaključavanja određuju ponašanje sistema u okruženju sa konkurentnim pristupom [3].

Ovaj seminarski rad ima za cilj da pruži pregled ključnih aspekata obrade transakcija u MongoDB-u, uključujući jednodokumentnu i višedokumentnu atomičnost, analizu planova izvršavanja upita, nivoe čitanja i pisanja (read/write concern), mehanizme izolacije, kao i strategije zaključavanja na različitim nivoima granularnosti.

2 Model podataka i atomičnost operacija

2.1 Dokumentni model i jednodokumentna atomičnost

MongoDB koristi dokumentni model podataka gde se podaci skladište u BSON dokumentima organizovanim u kolekcije. Osnovno svojstvo MongoDB-a je da su sve operacije nad pojedinačnim dokumentom atomične - uključujući operacije nad ugnježdenim dokumentima i nizovima unutar jednog dokumenta [1].

Ova jednodokumentna atomičnost predstavlja fundamentalnu razliku u pristupu u odnosu na relacione baze podataka. Dok relacioni model zahteva normalizaciju podataka u više tabela (što nužno zahteva transakcije za održavanje konzistentnosti), dokumentni model omogućava denormalizaciju i ugnježdavanje povezanih podataka u jedan dokument, čime se mnogi scenariji koji bi u relacionoj bazi zahtevali transakcije mogu rešiti jednom atomičnom operacijom [2].

```
// Atomično ažuriranje ugnježdenog dokumenta i niza
db.narudzbine.updateOne(
  { _id: ObjectId("64a1b2c3d4e5f6a7b8c9d0e1") },
  {
    $set: {
      "status": "isporucena",
      "isporuka.datum": new Date(),
      "isporuka.adresa": "Bulevar Nemanjića 12, Niš"
    },
    $push: {
      "istorija": {
        akcija: "promena_statusa",
        datum: new Date(),
        stari_status: "u_pripremi",
        novi_status: "isporucena"
      }
    },
    $inc: { "brojAzuriranja": 1 }
  }
);
```

Listing 1: Atomična operacija nad jednim dokumentom

```
  {
  "_id": 1,
  "kupac_id": "K001",
  "kupac": "Petar Petrovic",
  "datum": {
    "$date": "2025-01-20T00:00:00.000Z"
  },
  "status": "isporucena",
  "stavke": [
    {
      "proizvod_sifra": "LAP-001",
      "proizvod": "Laptop Lenovo ThinkPad",
      "kolicina": 1,
      "cena": 85000
    },
    {
      "proizvod_sifra": "MIS-001",
      "proizvod": "Mis Logitech MX",
      "kolicina": 2,
```

```
      "cena": 4500
    }
  ],
  "iznos": 94000,
  "ukupno": 94000,
  "brojAzuriranja": 1,
  "isporuka": {
    "adresa": "Bulevar Nemanjića 12, Niš",
    "datum": {
      "$date": "2026-04-16T20:15:19.245Z"
    }
  },
  "istorija": [
    {
      "akcija": "promena_statusa",
      "datum": {
        "$date": "2026-04-16T20:15:19.245Z"
      },
      "stari_status": "u_pripremi",
      "novi_status": "isporucena"
    }
  ]
}
```

Listing 2: Podaci nakon operacije azuriranja

U prethodnom primeru, operacija `updateOne` atomično menja status narudžbine, dodaje informacije o isporuci, dodaje zapis u istoriju promena i inkrementira brojač - sve kao jedna nedeljiva operacija, bez potrebe za eksplicitnom transakcijom.

2.2 Modelovanje podataka za smanjenje potrebe za transakcijama

MongoDB preporučuje modelovanje podataka koje maksimizuje korišćenje jednodokumentne atomičnosti. Ključne strategije uključuju [1]:

- **Ugnježdavanje (embedding)** - Povezani podaci koji se uvek zajedno čitaju i menjaju treba da budu ugnježdeni u isti dokument. Na primer, stavke narudžbine unutar dokumenta narudžbine
- **Denormalizacija** - Dupliciranje često čitanih podataka u dokument eliminiše potrebu za spajanjem više kolekcija
- **Atomični operatori** - Korišćenje operatora kao što su `$set`, `$inc`, `$push`, `$pull` za atomične modifikacije delova dokumenta

```
// Relacioni pristup: dve kolekcije, zahteva transakciju
// kolekcija: narudzbine
{ _id: 1, kupac: "Petar", ukupno: 5000 }
// kolekcija: stavke_narudzbine
{ narudzbina_id: 1, proizvod: "Laptop", cena: 5000 }

// Dokumentni pristup: jedan dokument, atomična operacija
db.narudzbine.insertOne({
  _id: 1,
```

```
  kupac: "Petar",
  datum: new Date("2025-03-15"),
  status: "nova",
  stavke: [
    { proizvod: "Laptop", kolicina: 1, cena: 5000 },
    { proizvod: "Miš", kolicina: 2, cena: 500 }
  ],
  ukupno: 6000
});
```

Listing 3: Modelovanje podataka - ugnježdavanje umesto razdvajanja

Ipak, postoje scenariji gde ugnježdavanje nije praktično ili moguće - na primer, kada su dokumenti previše veliki (MongoDB ograničava veličinu dokumenta na 16 MB), kada se ugnježdeni podaci nezavisno ažuriraju, ili kada postoje veze tipa n:m. U tim slučajevima potrebno je koristiti višedokumentne transakcije [4].

3 Višedokumentne ACID transakcije

3.1 ACID svojstva u MongoDB-u

Od verzije 4.0, MongoDB podržava višedokumentne transakcije sa punim ACID svojstvima [2]:

- **Atomičnost (Atomicity)** - Transakcija se izvršava u celosti ili se sve promene poništavaju. Ako bilo koja operacija unutar transakcije ne uspe, sve prethodne promene se vraćaju u prethodno stanje
- **Konzistentnost (Consistency)** - Transakcija prevodi bazu iz jednog konzistentnog stanja u drugo. Sva ograničenja definisana shemom (schema validation) moraju biti zadovoljena
- **Izolacija (Isolation)** - Operacije unutar transakcije su izolovane od konkurentnih operacija. MongoDB koristi snapshot izolaciju koja obezbeđuje da transakcija vidi konzistentan pogled na podatke
- **Trajnost (Durability)** - Nakon potvrde transakcije (commit), promene su trajno sačuvane čak i u slučaju pada sistema, u zavisnosti od konfiguracije write concern-a

3.2 Pokretanje i upravljanje transakcijama

Transakcije u MongoDB-u se pokreću kroz sesiju (session). Tok rada uključuje kreiranje sesije, pokretanje transakcije, izvršavanje operacija i potvrdu ili poništavanje transakcije [2].

```
// Pokretanje sesije i transakcije
const session = db.getMongo().startSession()

await session.withTransaction(() => {
  const col = session.getDatabase('test-db').getCollection('racuni')
  const alice = col.findOne({ _id: 'alice' })
  if (alice.stanje < 300) throw new Error('Nedovoljan saldo')
  col.updateOne({ _id: 'alice' }, { $inc: { stanje: -300 } })
  col.updateOne({ _id: 'bob'   }, { $inc: { stanje: +300 } })
})

session.endSession()
db.racuni.find()
```

Listing 4: Osnovno korišćenje višedokumentne transakcije

3.3 Callback API za transakcije

MongoDB pruža i Callback API (`withTransaction`) koji automatski upravlja ponovnim pokušajima u slučaju prolaznih grešaka (transient transaction errors) i grešaka prilikom potvrde (commit errors) [2].

```
const session = db.getMongo().startSession();

const transakcija = session.withTransaction(function() {
  const racuni = session.getDatabase("banka").racuni;
```

```
  const transakcije = session.getDatabase("banka").transakcije;

  // Transfer sredstava
  racuni.updateOne(
    { _id: "racun_A", stanje: { $gte: 500 } },
    { $inc: { stanje: -500 } },
    { session }
  );

  racuni.updateOne(
    { _id: "racun_B" },
    { $inc: { stanje: 500 } },
    { session }
  );

  // Zapis transakcije u log
  transakcije.insertOne({
    sa_racuna: "racun_A",
    na_racun: "racun_B",
    iznos: 500,
    datum: new Date(),
    status: "zavrsena"
  }, { session });

}, {
  readConcern: { level: "snapshot" },
  writeConcern: { w: "majority" }
});

session.endSession();
```

Listing 5: Callback API sa automatskim ponovnim pokušajima

Callback API automatski ponavlja celu transakciju ako dođe do `TransientTransactionError` (na primer zbog konflikta prilikom pisanja) i ponavlja `commitTransaction` ako dođe do `UnknownTransactionCommitResult` greške [1].

3.4 Ograničenja višedokumentnih transakcija

Višedokumentne transakcije u MongoDB-u imaju određena ograničenja koja treba uzeti u obzir [2]:

- **Vremensko ograničenje** - Podrazumevano, transakcija mora biti završena u roku od 60 sekundi (`transactionLifetimeLimitSeconds`). Duže transakcije se automatski poništavaju
- **Veličina oplog zapisa** - Ukupna veličina svih promena u transakciji ne sme premašiti 16 MB oplog zapisa
- **DDL operacije** - Kreiranje ili brisanje kolekcija i indeksa nije dozvoljeno unutar transakcije (od verzije 4.4 kreiranje kolekcije je podržano implicitno)
- **Performanse** - Transakcije uvode dodatni overhead (zaključavanje, snapshot upravljanje, oplog pisanje), pa ih treba koristiti samo kada je to neophodno
- **Write conflict** - Ako dve transakcije pokušaju da modifikuju isti dokument, druga transakcija će dobiti write conflict grešku i moraće da se ponovi

3.5 Savepoints i načini pristupa transakcijama

Za razliku od relacionih baza podataka koje podržavaju tačke čuvanja (*savepoints*) i delimični rollback, MongoDB ne podržava savepoints unutar transakcija. Transakcija se u celosti potvrđuje ili u celosti poništava - nema mogućnosti vraćanja izmena do određene međutačke. U praksi to znači da logiku koja bi u SQL-u koristila savepoints treba prestrukturirati u više manjih, nezavisnih transakcija ili eliminisati potrebu za njima [2].

MongoDB takođe ne definiše eksplicitni *access mode* za transakcije na način koji to čini SQL standard (`READ ONLY` / `READ WRITE`). Umesto toga, ponašanje se kontroliše kombinacijom `readPreference` i `readConcern` parametara. Važno ograničenje je da transakcije uvek moraju čitati sa primarnog čvora - čitanje sa sekundarnih čvorova unutar aktivne transakcije nije podržano:

```
// Unutar transakcije - readPreference mora biti "primary"
session.startTransaction({
  readConcern: { level: "snapshot" },
  writeConcern: { w: "majority" },
  readPreference: "primary"  // jedina validna opcija unutar transakcije
});

// Van transakcije - čitanje sa sekundarnog čvora
// Ekvivalent READ ONLY scenarija - rasterećuje primarni čvor
db.narudzbine.find(
  { status: "aktivna" },
  { readPreference: "secondaryPreferred" }
);
```

Listing 6: Kontrola načina pristupa kroz parametre sesije

3.6 Distribuirane transakcije na sharded klasterima

Od verzije 4.2, MongoDB podržava ACID transakcije koje obuhvataju više shard-ova u sharded klasteru. Ovakve transakcije se izvršavaju kroz protokol dvofaznog commit-a (*two-phase commit, 2PC*), koji obezbeđuje da se transakcija atomično potvrdi ili poništi na svim uključenim shard-ovima [5].

Ključne komponente distribuiranog izvršavanja su:

- **Transaction Coordinator** - Jedan od shard-ova (obično onaj na kom je pokrenuta prva operacija pisanja) preuzima ulogu koordinatora. Koordinator prati učesnike transakcije i upravlja 2PC protokolom kroz internu `config.transaction_coordinators` kolekciju

- **Prepare faza** - Koordinator šalje `prepareTransaction` komandu svakom učesničkom shard-u. Svaki učesnik priprema transakciju, drži lock-ove nad modifikovanim dokumentima i odgovara pozitivno (*prepared*) ili negativno (*abort*)

- **Commit/Abort faza** - Ako su svi učesnici odgovorili pozitivno, koordinator šalje `commitTransaction` svim učesnicima. U slučaju bilo koje negativne potvrde ili isteka vremena, šalje `abortTransaction`

- **Logičko vreme i kauzalna konzistentnost** - Za redosled operacija preko shard-ova MongoDB koristi hibridni logički sat, što omogućava kauzalnu konzistentnost i snapshot čitanje u distribuiranom kontekstu [6]

```
// Sesija transparentno radi preko sharded klastera kroz mongos
const session = db.getMongo().startSession();

session.startTransaction({
  readConcern: { level: "snapshot" },
  writeConcern: { w: "majority" }
});

try {
  // Dokument kupca (shard ključ: kupac_id) - shard A
  session.getDatabase("prodavnica").kupci.updateOne(
    { _id: "K001" },
    { $inc: { bodovi_vernosti: 100 } },
    { session }
  );

  // Dokument narudzbine (shard ključ: narudzbina_id) - shard B
  session.getDatabase("prodavnica").narudzbine.insertOne(
    { _id: "N5001", kupac: "K001", iznos: 12000, datum: new Date() },
    { session }
  );

  // 2PC protokol se automatski izvršava pri commit-u
  session.commitTransaction();
} catch (e) {
  session.abortTransaction();
} finally {
  session.endSession();
}
```

Listing 7: Distribuirana transakcija preko više shard-ova

Distribuirane transakcije donose dodatne troškove u odnosu na transakcije na jednom replica set-u Preporuka je da se ključ shard-a modeluje tako da se povezani podaci nalaze na istom shard-u (*shard-locality*) i da se distribuirane transakcije koriste samo kada je to neizbežno [5].

4 Planovi izvršavanja upita

4.1 Query Optimizer u MongoDB-u

MongoDB koristi cost-based query optimizer koji bira najefikasniji plan izvršavanja za svaki upit. Optimizer analizira dostupne indekse, procenjuje cenu različitih strategija pristupa i bira plan sa najnižom procenjenom cenom [3].

Za razliku od relacionih baza podataka koje koriste statistike o distribuciji podataka, MongoDB optimizer koristi empirijski pristup - za nove upite, testira više kandidatskih planova paralelno u tzv. *trial period* fazi i bira plan koji prvi vrati zadati broj rezultata. Odabrani plan se kešira u Plan Cache za buduće izvršavanje istog oblika upita (query shape) [7].

4.2 Metoda explain()

Metoda `explain()` je osnovni alat za analizu planova izvršavanja u MongoDB-u. Pruža detaljan uvid u način na koji MongoDB obrađuje upit, uključujući izbor indeksa, broj pregledanih dokumenata i vreme izvršavanja [3].

Metoda `explain()` podržava tri nivoa detaljnosti:

- **queryPlanner** - Prikazuje odabrani plan i razmatrane kandidatske planove bez stvarnog izvršavanja upita
- **executionStats** - Izvršava upit i prikazuje statistike izvršavanja (broj pregledanih dokumenata, vreme, itd.)
- **allPlansExecution** - Izvršava sve kandidatske planove i prikazuje uporedne statistike

```
// Osnovni explain - queryPlanner mode
db.narudzbine.find(
  { kupac_id: "K001", status: "aktivna" }
).explain("queryPlanner");

// Explain sa statistikama izvršavanja
db.narudzbine.find(
  { kupac_id: "K001", status: "aktivna" }
).sort({ datum: -1 }).explain("executionStats");

// Explain za agregacioni pipeline
db.narudzbine.explain("executionStats").aggregate([
  { $match: { datum: { $gte: new Date("2025-01-01") } } },
  { $group: {
      _id: "$kupac_id",
      ukupno: { $sum: "$iznos" },
      broj: { $sum: 1 }
  }},
  { $sort: { ukupno: -1 } },
  { $limit: 10 }
]);
```

Listing 8: Korišćenje explain() za analizu upita

```
executionStats: {
  executionSuccess: true,
  nReturned: 1,
  executionTimeMillis: 0,
  totalKeysExamined: 0,
  totalDocsExamined: 5,
  executionStages: {
    isCached: false,
    stage: 'SORT',
    nReturned: 1,
    executionTimeMillisEstimate: 0,
    works: 8,
    advanced: 1,
    needTime: 6,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    sortPattern: { datum: -1 },
    memLimit: 104857600,
    type: 'simple',
    totalDataSizeSorted: 278,
    usedDisk: false,
    spills: 0,
    spilledRecords: 0,
    spilledBytes: 0,
    spilledDataStorageSize: 0,
    inputStage: {
      stage: 'COLLSCAN',
      filter: {
        '$and': [
          { kupac_id: { '$eq': 'K001' } },
          { status: { '$eq': 'aktivna' } }
        ]
      },
      nReturned: 1,
      executionTimeMillisEstimate: 0,
      works: 6,
      advanced: 1,
      needTime: 4,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      direction: 'forward',
      docsExamined: 5
    }
  }
},
```


Slika 1: Prikaz statistike izvršavanja transakcija

4.3 Ključne faze plana izvršavanja

Plan izvršavanja u MongoDB-u se sastoji od niza faza (stages) koje opisuju korake obrade upita. Najvažnije faze su [7]:

Tabela 1: Ključne faze plana izvršavanja

Faza (Stage)	**Opis**
COLLSCAN	Sekvencijalno skeniranje cele kolekcije (bez indeksa)
IXSCAN	Skeniranje indeksa - pretraga kroz B-tree strukturu indeksa
FETCH	Dobavljanje kompletnih dokumenata na osnovu pokazivača iz indeksa
SORT	Sortiranje rezultata u memoriji (ograničeno na 100 MB)
SORT_KEY_GENERATOR	Generisanje ključeva za sortiranje
PROJECTION_COVERED	Projekcija koja potpuno upotrebljava indekse (covered query)
LIMIT	Ograničavanje broja rezultata
SKIP	Preskakanje zadatog broja rezultata
COUNT_SCAN	Brojanje korišćenjem indeksa bez pristupa dokumentima

4.4 Analiza efikasnosti upita

Ključne metrike iz `executionStats` rezultata koje ukazuju na efikasnost upita su [3]:

- **totalDocsExamined** - Ukupan broj pregledanih dokumenata. Idealno, ovaj broj treba da bude blizak broju vraćenih dokumenata (`nReturned`)
- **totalKeysExamined** - Ukupan broj pregledanih ključeva indeksa. Visok odnos `totalKeysExamined / nReturned` ukazuje na neefikasno korišćenje indeksa
- **executionTimeMillis** - Ukupno vreme izvršavanja upita u milisekundama
- **stage: COLLSCAN** - Prisustvo faze COLLSCAN ukazuje na to da upit ne koristi indeks i zahteva pregled cele kolekcije

```
// Upit bez indeksa - COLLSCAN
db.narudzbine.find({ status: "aktivna" }).explain("executionStats");
// Rezultat: stage: "COLLSCAN", totalDocsExamined: 1000000

// Kreiranje odgovarajućeg indeksa
db.narudzbine.createIndex({ status: 1, datum: -1 });

// Isti upit sa indeksom - IXSCAN + FETCH
db.narudzbine.find({ status: "aktivna" }).explain("executionStats");

// Covered query - samo IXSCAN, bez FETCH
db.narudzbine.find(
  { status: "aktivna" },
  { _id: 0, status: 1, datum: 1 }
).explain("executionStats");
```

Listing 9: Analiza efikasnosti upita na osnovu explain rezultata

```
executionStats: {
  executionSuccess: true,
  nReturned: 0,
  executionTimeMillis: 0,
  totalKeysExamined: 0,
  totalDocsExamined: 5,
  executionStages: {
    isCached: false,
    stage: 'COLLSCAN',
    filter: { status: { '$eq': ' aktivna ' } },
    nReturned: 0,
    executionTimeMillisEstimate: 0,
    works: 6,
    advanced: 0,
    needTime: 5,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    direction: 'forward',
    docsExamined: 5
  }
},
```

Slika 2: Prikaz statistike izvršavanja operacije sa COLLSCAN

```
executionStats: {
  executionSuccess: true,
  nReturned: 2,
  executionTimeMillis: 8,
  totalKeysExamined: 2,
  totalDocsExamined: 2,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 2,
    executionTimeMillisEstimate: 0,
    works: 3,
    advanced: 2,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    docsExamined: 2,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 2,
      executionTimeMillisEstimate: 0,
      works: 3,
      advanced: 2,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
```

Slika 3: Prikaz statistike izvršavanja operacije sa IXSCAN + FETCH

```
winningPlan: {
  isCached: false,
  queryPlan: {
    stage: 'GROUP',
    planNodeId: 4,
    inputStage: {
      stage: 'FETCH',
      planNodeId: 2,
      inputStage: {
        stage: 'IXSCAN',
        planNodeId: 1,
        keyPattern: { status: 1, datum: -1 },
        indexName: 'status_1_datum_-1',
        isMultiKey: false,
        multiKeyPaths: { status: [], datum: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: {
          status: [ '["zavrsena", "zavrsena"]' ],
          datum: [
            '[new Date(9223372036854775807), new Date(1735689600000)]'
          ]
        }
      }
    }
  },
```

Slika 4: Prikaz statistike izvršavanja operacije sa IXSCAN bez FETCH

4.5 Plan Cache

MongoDB kešira odabrane planove izvršavanja u Plan Cache kako bi izbegao ponavljanje faze evaluacije za upite istog oblika. Plan Cache se automatski invalidira u sledećim situacijama [7]:

- Kreiranje ili brisanje indeksa na kolekciji
- Restartovanje `mongod` procesa
- Kada keširani plan postane neefikasan (MongoDB automatski reevaluira plan ako broj pregledanih dokumenata značajno premašuje očekivani broj)

```
// Pregled planova u kešu za kolekciju
db.narudzbine.getPlanCache().list();

// Brisanje keša planova za kolekciju
db.narudzbine.getPlanCache().clear();

// Brisanje keša za specifičan oblik upita
db.narudzbine.getPlanCache().clearPlansByQuery(
  { status: "aktivna" },
  { datum: -1 },
  { _id: 0, status: 1 }
);
```

Listing 10: Upravljanje Plan Cache-om

4.6 Planovi agregacionog pipeline-a

Agregacione upite MongoDB izvršava kao niz povezanih faza (stages) u pipeline-u, gde izlaz jedne faze postaje ulaz naredne. Metoda `explain()` za agregacije vraća strukturu u kojoj se za svaku fazu prikazuju posebni statistički podaci - za razliku od `find()` upita gde postoji jedan stablasti plan, pipeline plan je linearni niz [8].

Optimizator pipeline-a automatski primenjuje nekoliko transformacija pre izvršavanja:

- **Predicate pushdown** - `$match` faze se pomeraju što je bliže izvoru dokumenata, omogućavajući filtriranje preko indeksa u IXSCAN fazi umesto u memoriji
- **Sort-merge optimizacija** - Ako postoji kompozitni pokrivajuci indeks i filter (`$match`) i redosled (`$sort`), blokirajuća SORT faza se eliminiše
- **Projection pushdown** - `$project` i `$unset` faze koje odbacuju polja prenose se na gore, smanjujući obim dokumenata u narednim fazama
- **Spajanje uzastopnih faza** - Dve uzastopne `$match` faze spajaju se u jednu logičkom konjunkcijom; `$limit` posle `$sort` koristi se za *top-k* optimizaciju

```
// Pipeline sa filtrom, grupisanjem, sortiranjem i limitom
db.narudzbine.aggregate([
  { $match: { status: "zavrsena",
              datum: { $gte: ISODate("2025-01-01") } } },
  { $group: { _id: "$kupac_id",
              ukupno: { $sum: "$iznos" },
              broj: { $sum: 1 } } },
  { $sort: { ukupno: -1 } },
  { $limit: 10 }
]).explain("executionStats");

// U rezultatu se za svaku fazu prikazuje:
```

```
// - stage:            tip faze (IXSCAN, COLLSCAN, GROUP, SORT, LIMIT)
// - nReturned:        broj dokumenata koji prolaze kroz fazu
// - executionTimeMillisEstimate: procenjeno vreme faze
```

Listing 11: Analiza plana agregacionog pipeline-a

Za performanse pipeline-a ključna su dva ograničenja [8]:

- **Memorijski limit po fazi** - Svaka blokirajuća faza (`$group`, `$sort`, `$bucket`) ima podrazumevano 100 MB memorijski limit. Za veće agregacije potrebno je postaviti `allowDiskUse: true` što dozvoljava izlivanje na disk po cenu performansi
- **`$lookup` i indeksiranje** - `$lookup` (ekvivalent LEFT OUTER JOIN-a) efikasno radi samo ako strana kolekcija ima indeks nad poljem sa kojim se join-uje. Bez indeksa, `$lookup` izvršava COLLSCAN za svaki ulazni dokument (kompleksnost $O(n \cdot m)$) što za velike kolekcije postaje neupotrebljivo

```
// Indeks na stranoj kolekciji pretvara $lookup iz
// COLLSCAN po ulaznom dokumentu u IXSCAN
db.proizvodi.createIndex({ sifra: 1 });

db.narudzbine.aggregate([
  { $match: { status: "aktivna" } },        // koristi indeks
  { $lookup: {
      from: "proizvodi",
      localField: "proizvod_sifra",
      foreignField: "sifra",                // indeksirano -> IXSCAN
      as: "proizvod"
  }},
  { $unwind: "$proizvod" },
  { $project: { kupac: 1, "proizvod.naziv": 1, iznos: 1 } }
], { allowDiskUse: true });
```

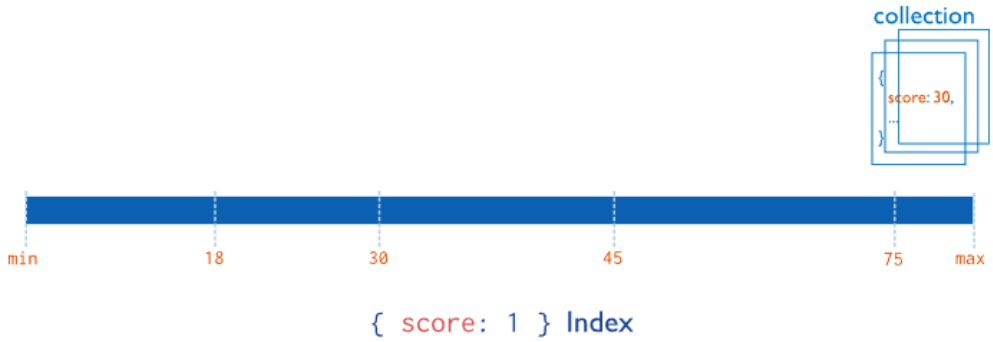
Listing 12: Optimizacija $lookup operacije kroz indeks

5 Indeksi i optimizacija planova izvršavanja

5.1 Tipovi indeksa u MongoDB-u

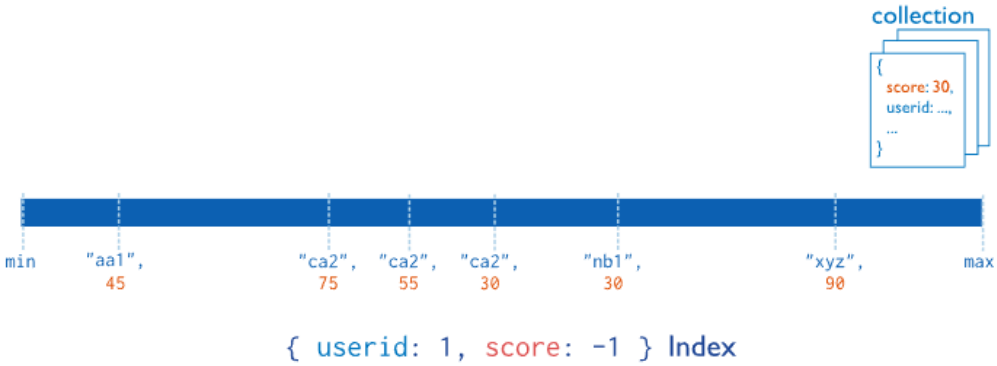
MongoDB podržava više tipova indeksa koji direktno utiču na planove izvršavanja [9]:

- **Jednostruki indeks (Single Field)** - Indeks nad jednim poljem dokumenta. Podrazumevano, MongoDB kreira indeks nad poljem `_id`



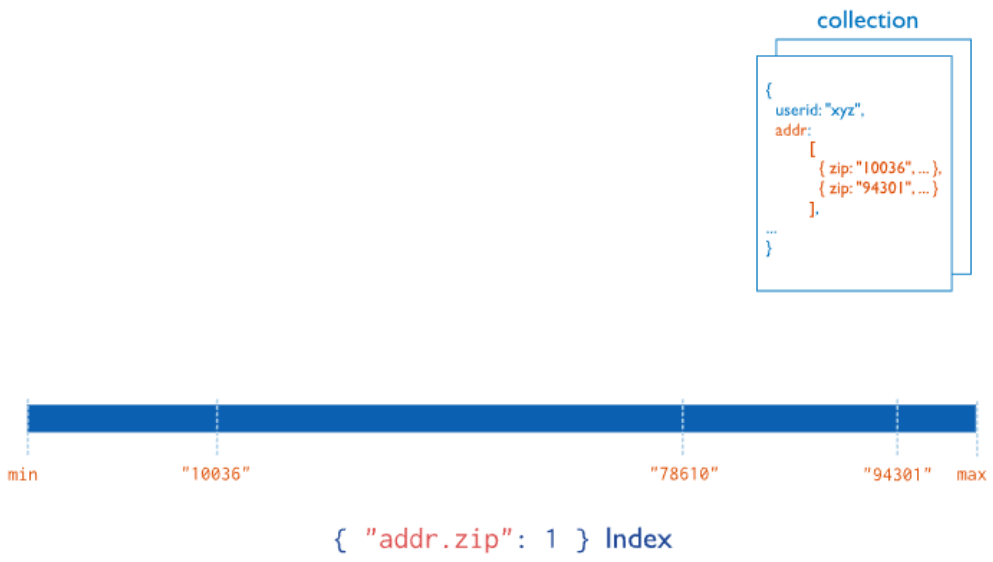
Slika 5: Prikaz skladistenja jednostrukog indeksa

- **Kompozitni indeks (Compound)** - Indeks nad više polja. Redosled polja je važan za efikasnost, slično kao kod B-tree indeksa u relacionim bazama



Slika 6: Prikaz skladistenja kompozitnog kljuca

- **Multikey indeks** - Automatski se kreira kada se indeksira polje koje sadrži niz vrednosti. Za svaki element niza kreira se zaseban unos u indeksu



Slika 7: Prikaz skladistenja multikey indeksa

- **Tekstualni indeks (Text)** - Omogućava pretragu teksta po celim rečima (full-text search)
- **Geospacijalni indeks (2dsphere, 2d indeksi)** - Indeksi za geografske podatke i upite
- **Hashed indeks** - Indeksira heš vrednost polja, koristan za sharding. Hesirani indeksi ne mogu da budu pokrivajuci

```
//Kreiranje jednostrukog hesiranog indeksa
db.<collection>.createIndex(
  {
    <field>: "hashed"
  }
//Kreiranje hesiranog indeksa nad vise polja
db.<collection>.createIndex(
{
    <field1>: "hashed",
    <field2>: "<sort order>",
    <field3>: "<sort order>",
    ...
})
)
```

Listing 13: Kreiranje hesiranog indeksa

- **Wildcard indeks** - Indeksira sva polja ili podskup polja u dokumentu, koristan za sheme sa promenljivom strukturom

```
          db.narudzbine.createIndex({ "$**": 1 });

```

Listing 14: Kreiranje wildcard indeksa

```
// Jednostruki indeks
db.narudzbine.createIndex({ kupac_id: 1 });

// Kompozitni indeks (redosled je važan!)
db.narudzbine.createIndex({ status: 1, datum: -1, kupac_id: 1 });

// Multikey indeks nad poljem koje sadrži niz
db.proizvodi.createIndex({ "tagovi": 1 });

// Parcijalni indeks (ekvivalent filtriranog indeksa)
db.narudzbine.createIndex(
  { datum: -1 },
  { partialFilterExpression: { status: "aktivna" } }
);

// TTL indeks - automatsko brisanje dokumenata
db.sesije.createIndex(
  { "poslednja_aktivnost": 1 },
  { expireAfterSeconds: 3600 }
);

// Unique indeks
db.korisnici.createIndex(
  { email: 1 },
  { unique: true }
);
```

Listing 15: Kreiranje različitih tipova indeksa

5.2 ESR pravilo za kompozitne indekse

Prilikom projektovanja kompozitnih indeksa, preporučuje se ESR pravilo (Equality, Sort, Range) koje definiše optimalan redosled polja [3]:

1. **Equality (E)** - Polja koja se koriste u uslovima jednakosti (`$eq`) treba da budu prva u indeksu
2. **Sort (S)** - Polja po kojima se sortira treba da budu sledeća, čime se eliminiše potreba za sortiranjem u memoriji
3. **Range (R)** - Polja koja se koriste u uslovima opsega (`$gt`, `$lt`, `$gte`, `$lte`) treba da budu poslednja

```
// Upit:
db.narudzbine.find({
  status: "aktivna",          // Equality
  iznos: { $gte: 1000 }       // Range
}).sort({ datum: -1 });        // Sort

// Optimalan indeks po ESR pravilu:
// E(status) -> S(datum) -> R(iznos)
db.narudzbine.createIndex({ status: 1, datum: -1, iznos: 1 });

// Loš redosled - Range pre Sort:
db.narudzbine.createIndex({ status: 1, iznos: 1, datum: -1 });
```

Listing 16: Primena ESR pravila

5.3 Hint i forsiranje plana izvršavanja

Metoda `hint()` omogućava eksplicitno navođenje indeksa koji MongoDB treba da koristi za dati upit, zaobilazeći odluku optimizatora [1]:

```
// Forsiranje korišćenja specifičnog indeksa
db.narudzbine.find({ kupac_id: "K001", status: "aktivna" })
  .hint({ kupac_id: 1, status: 1 })
  .explain("executionStats");

// Forsiranje COLLSCAN (za testiranje)
db.narudzbine.find({ kupac_id: "K001" })
  .hint({ $natural: 1 })
  .explain("executionStats");
```

Listing 17: Korišćenje hint() za forsiranje indeksa

6 Read Concern i Write Concern

6.1 Write Concern

Write Concern definiše nivo garancije da je operacija pisanja uspešno zabeležena. Određuje koliko članova replica set-a mora potvrditi prijem operacije pre nego što se klijentu vrati uspešan odgovor [10].

Tabela 2: Nivoi Write Concern-a

Write Concern	Opis
`w: 0`	Bez potvrde - klijent ne čeka potvrdu. Najbrže, ali bez garancije
`w: 1`	Potvrda od primarnog čvora (podrazumevano). Pisanje je u memoriji
`w: "majority"`	Potvrda od većine članova replica set-a. Obezbeđuje trajnost i otpornost na gubitak primarnog čvora
`w: <broj>`	Potvrda od zadatog broja članova replica set-a
`j: true`	Zahteva pisanje u journal pre potvrde, obezbeđujući trajnost čak i u slučaju pada sistema

```
// Pisanje sa potvrdom većine - visoka trajnost
db.narudzbine.insertOne(
  { kupac: "Ana", iznos: 3500, datum: new Date() },
  { writeConcern: { w: "majority", j: true } }
);

// Pisanje bez čekanja na potvrdu - visoke performanse
db.logovi.insertOne(
  { poruka: "Korisnik se prijavio", vreme: new Date() },
  { writeConcern: { w: 0 } }
);

// Podrazumevani write concern na nivou kolekcije
db.createCollection("kriticni_podaci", {
  writeConcern: { w: "majority", j: true }
});
```

Listing 18: Korišćenje različitih Write Concern nivoa

6.2 Read Concern

Read Concern definiše nivo konzistentnosti i izolacije podataka koji se čitaju. Određuje koji podaci su vidljivi operaciji čitanja u kontekstu replikacije [11].

Tabela 3: Nivoi Read Concern-a

Read Concern	Opis
`local`	Vraća najnovije podatke dostupne na čvoru, bez garancije da su replicirani. Podrazumevani nivo
`available`	Slično kao local, ali za sharded kolekcije može vratiti orphaned dokumente tokom migracije chunk-ova
`majority`	Vraća samo podatke koji su potvrđeni od većine članova replica set-a. Garantuje trajnost pročitanih podataka
`linearizable`	Garantuje da čitanje odražava sve uspešno potvrđene operacije pisanja pre početka čitanja. Najstroži nivo
`snapshot`	Vraća podatke iz snapshot-a većine u određenom trenutku. Koristi se u višedokumentnim transakcijama

```
// Čitanje sa garancijom trajnosti
db.racuni.find(
  { _id: "racun_A" }
).readConcern("majority");

// Linearizable čitanje - najstroža konzistentnost
db.racuni.find(
  { _id: "racun_A" }
).readConcern("linearizable");

// Snapshot čitanje unutar transakcije
session.startTransaction({
  readConcern: { level: "snapshot" },
  writeConcern: { w: "majority" }
});
```

Listing 19: Korišćenje različitih Read Concern nivoa

6.3 Kauzalna konzistentnost

MongoDB podržava kauzalno konzistentne sesije koje garantuju da operacije koje su kauzalno povezane budu viđene u ispravnom redosledu. Ovo je posebno važno u distribuiranim okruženjima sa čitanjem sa sekundarnih čvorova [12]. Implementacija se oslanja na klaster logički sat (*cluster-wide logical clock*) i hibridni logički sat koji se distribuira uz svaku operaciju, čime se redosled kauzalno povezanih događaja održava čak i između različitih čvorova replica set-a [6].

```
// Kreiranje kauzalno konzistentne sesije
const session = db.getMongo().startSession({ causalConsistency: true });
const kolekcija = session.getDatabase("prodavnica").narudzbine;

// Pisanje na primarnom čvoru
kolekcija.insertOne(
  { _id: "N100", kupac: "Marko", iznos: 2500 },
  { session }
);

// Čitanje sa sekundarnog čvora
kolekcija.find(
  { _id: "N100" },
```

```
  { session, readPreference: "secondary" }
);

session.endSession();
```

Listing 20: Kauzalno konzistentna sesija

7 Izolacija transakcija

7.1 Snapshot izolacija

MongoDB koristi snapshot izolaciju za višedokumentne transakcije. Kada transakcija započne sa `readConcern: ''snapshot''`, ona vidi konzistentan snimak (snapshot) podataka od trenutka početka transakcije. Sve promene koje druge transakcije naprave nakon tog trenutka nisu vidljive unutar tekuće transakcije [2].

Tabela 4: Poređenje izolacije u MongoDB i relacionim bazama podataka

Anomalija	**MongoDB (snapshot)**	**SQL Server (Read Committed)**
Dirty Read	Sprečeno	Sprečeno
Non-repeatable Read	Sprečeno	Moguće
Phantom Read	Sprečeno	Moguće
Write Skew	Moguće	Sprečeno (Serializable)

Snapshot izolacija garantuje da transakcija neće videti dirty, non-repeatable ili phantom čitanja. Međutim, moguća je anomalija poznata kao write skew, gde dve transakcije čitaju iste podatke i na osnovu pročitanog donose odluke o pisanju koje su međusobno konfliktne [13].

7.2 Write Conflict i optimistična kontrola konkurentnosti

MongoDB koristi optimističnu kontrolu konkurentnosti za transakcije. Ako dve transakcije pokušaju da modifikuju isti dokument, dolazi do write conflict-a [2]:

1. Prva transakcija koja zapiše/azurira dokument "zaključava" taj dokument
2. Druga transakcija koja pokuša da azurira isti dokument dobija `WriteConflict` grešku
3. Druga transakcija mora da se ponovi (retry) od početka

```
// Transakcija 1: čita i ažurira dokument
const session1 = db.getSession().getClient().startSession();
const session2 = db.getSession().getClient().startSession();

session1.startTransaction();
db.proizvodi.updateOne(
  { _id: "P001" },
  { $inc: { stanje: -5 } },
  { session: session1 }
```

```
);
// Transakcija 1 još nije potvrdila...

// Transakcija 2: pokušava da ažurira isti dokument
session2.startTransaction();
db.proizvodi.updateOne(
  { _id: "P001" },
  { $inc: { stanje: -3 } },
  { session: session2 }
);
// Rezultat: MongoServerError: WriteConflict error: this operation
    conflicted  with another operation. Please retry your operation or
    multi-document transaction.
```

Listing 21: Write conflict scenario

Transakcija 2 mora da se ponovi nakon što Transakcija 1 završi (commit ili abort).

7.3 Izgubljena ažuriranja (Lost Updates)

Izgubljeno ažuriranje (*lost update*) nastaje kada dve konkurentne operacije pročitaju istu vrednost, samostalno je modifikuju i upišu nazad - ažuriranje jedne operacije prepisuje ažuriranje druge. Van eksplicitnih transakcija, MongoDB pruža zaštitu kroz atomične operatore koji eliminišu potrebu za read-modify-write ciklusom:

```
// Los primer: read-write van transakcije
// Ako dve instance izvrše ovo paralelno, jedna izmena se gubi
const p = db.proizvodi.findOne({ _id: "P001" });
db.proizvodi.updateOne(
  { _id: "P001" },
  { $set: { stanje: p.stanje - 5 } }  // <-- lost update rizik
);

// Dobar primer: atomični $inc - server izvršava inkrementiranje
// bez read-write, garantovano bez lost update-a
db.proizvodi.updateOne(
  { _id: "P001" },
  { $inc: { stanje: -5 } }
);

// Dodatna zaštita: uslovno ažuriranje samo ako ima dovoljno novca na
    stanju  (optimistička provera)
db.proizvodi.updateOne(
  { _id: "P001", stanje: { $gte: 5 } },
  { $inc: { stanje: -5 } }
);
```

Listing 22: Razlika između podložnog i atomičnog ažuriranja

7.4 Serijalizabilnost i rasporedi izvršavanja

U bazama podataka, raspored (*schedule*) se definiše kao redosled isprepletenih operacija iz više konkurentnih transakcija. Raspored je *serijalizabilan* ako je ekvivalentan nekom serijskom rasporedu (gde transakcije teku jedna za drugom bez preklapanja). Serijalizabilnost je najstroži praktični nivo izolacije koji garantuje odsustvo svih anomalija.

MongoDB-ova snapshot izolacija postiže nivo između `REPEATABLE READ` i `SERIALIZABLE` iz SQL standarda. Kombinacijom snapshot-a i write conflict detekcije, sprečene su sve klasične anomalije čitanja, ali ostaje mogućnost *write skew*-a - situacije gde dve transakcije čitaju iste podatke, donose odluke zasnovane na pročitanom, i pišu promene koje su međusobno nekonzistentne, a nijedna od operacija nije direktan konflikt:

Tabela 5: Položaj MongoDB izolacije u odnosu na SQL standard

Nivo izolacije	**Dirty Read**	**Lost Update**	**NR Read**	**Phantom**	**Write Skew**
Read Uncommitted	Da	Da	Da	Da	Da
Read Committed	Ne	Da	Da	Da	Da
Repeatable Read	Ne	Ne	Ne	Da	Da
MongoDB (Snapshot)	Ne	Ne	Ne	Ne	Da
Serializable	Ne	Ne	Ne	Ne	Ne

Za aplikacije koje ne smeju tolerisati write skew (npr. rezervacija istog resursa od strane dve transakcije), preporučena strategija u MongoDB-u je da obe transakcije modifikuju isti dokument - čime nastaje write conflict koji primorava serijalizaciju. Alternativno, modelovanje koje ugnježdava sve relevantne podatke u jedan dokument prirodno eliminiše ovaj problem [12].

8 Zaključavanje u MongoDB-u

8.1 WiredTiger Storage Engine i zaključavanje

Od verzije 3.2, MongoDB koristi WiredTiger kao podrazumevani storage engine [14]. WiredTiger implementira zaključavanje na nivou dokumenta (document-level locking), što predstavlja značajno poboljšanje u odnosu na prethodni MMAPv1 engine koji je koristio zaključavanje na nivou kolekcije [15].

MongoDB koristi višeslojni sistem zaključavanja sa sledećom hijerarhijom:

1. **Globalni nivo (Global lock)** - Zaključava celu instancu. Koristi se za administrativne operacije
2. **Nivo baze podataka (Database lock)** - Zaključava pojedinačnu bazu
3. **Nivo kolekcije (Collection lock)** - Zaključava kolekciju
4. **Nivo dokumenta (Document lock)** - Zaključava pojedinačni dokument (WiredTiger nivo)

8.2 Tipovi zaključavanja

MongoDB koristi intent lock-ove za efikasno upravljanje hijerarhijskim zaključavanjem [15]:

Tabela 6: Tipovi lock-ova u MongoDB-u

Lock tip	Oznaka	Opis
Shared (S)	r	Deljeni lock za čitanje. Više operacija može istovremeno držati S lock
Exclusive (X)	w	Ekskluzivni lock za pisanje. Samo jedna operacija može držati X lock
Intent Shared (IS)	R	Namera za čitanje na nižem nivou hijerarhije
Intent Exclusive (IX)	W	Namera za pisanje na nižem nivou hijerarhije

Kompatibilnost lock-ova je sledeća: IS je kompatibilan sa svim osim X lock-om; IX je kompatibilan sa IS i IX; S je kompatibilan sa IS i S; X nije kompatibilan ni sa jednim drugim lock-om.

8.3 Zaključavanje u praksi

Operacije čitanja na nivou MongoDB-a zahtevaju Intent Shared (IS) lock na nivou baze i kolekcije, dok WiredTiger interno čita dokument bez eksplicitnog lock-a korišćenjem MVCC (Multiversion Concurrency Control) mehanizma. Operacije pisanja zahtevaju Intent Exclusive (IX) lock na nivou baze i kolekcije, i ekskluzivni lock na nivou dokumenta [15].

```
// Pregled trenutnih operacija sa informacijama o lock-ovima
db.currentOp({
  "waitingForLock": true
});
```

```

// Pregled svih aktivnih operacija
db.currentOp({
  "active": true,
  "secs_running": { "$gt": 2 }
});

// Server Status - statistike zaključavanja
db.serverStatus().locks;

// Detaljne metrike zaključavanja
db.serverStatus().wiredTiger.concurrentTransactions;
```

Listing 23: Praćenje aktivnih lock-ova

8.4 Yielding - ustupanje lock-ova

MongoDB automatski ustupa (yield) lock-ove tokom dugotrajnih operacija kako bi omogućio drugim operacijama pristup resursima. Operacija čitanja ili pisanja koja pregledava veliku količinu dokumenata periodično oslobađa lock-ove, omogućavajući drugim operacijama da se izvrše [1].

Operacije koje ne ustupaju lock-ove:

- Operacije unutar višedokumentne transakcije (drže lock-ove do završetka transakcije)
- Administrativne operacije (npr. `compact`, `reIndex`)

8.5 Zaključavanje u višedokumentnim transakcijama

Višedokumentne transakcije drže lock-ove na svim dokumentima koje modifikuju tokom celog izvršenja transakcije (od početka do commit-a ili abort-a). Ovo znači da duže transakcije mogu blokirati druge operacije na istim dokumentima [2].

```
// Podešavanje maksimalnog vremena čekanja na lock (ms)
db.adminCommand({
  setParameter: 1,
  maxTransactionLockRequestTimeoutMillis: 5
});
// Podrazumevana vrednost je 5 ms
// Vrednost -1: neograničeno čekanje

// Podešavanje maksimalnog trajanja transakcije (sekunde)
db.adminCommand({
  setParameter: 1,
  transactionLifetimeLimitSeconds: 60
});
```

Listing 24: Konfiguracija vremenskog ograničenja za lock-ove

8.6 Deadlock i razrešavanje konflikata

U klasičnim relacionim bazama podataka, deadlock nastaje kada dve ili više transakcija cirkularno zavise jedna od druge u čekanju na lock-ove - na primer, transakcija

T_1 drži lock na resursu A i čeka B, dok T_2 drži B i čeka A. Klasični RDBMS sistemi (Oracle, PostgreSQL, SQL Server) detektuju takve cikluse u *wait-for* grafu i razrešavaju ih žrtvovanjem (*victim selection*) jedne od transakcija.

MongoDB ima drugačiji pristup - umesto graph-based detekcije ciklusa, oslanja se na **timeout-based razrešavanje** u kombinaciji sa optimističnom kontrolom konkurentnosti [2]:

- **Kratak timeout na lock zahtev** - Kroz `maxTransactionLockRequestTimeoutMillis` (podrazumevano 5 ms), ako transakcija ne može da pribavi lock u datom roku, odmah vraća `WriteConflict` grešku umesto da neograničeno čeka. Cirkularno čekanje je time eliminisano
- **Optimistična detekcija konflikta** - Prvi write nad dokumentom dobija lock; svaki naredni konkurentni write izaziva `WriteConflict` i primorava retry. Situacija analogna klasičnom deadlock-u razrešava se u roku od nekoliko milisekundi bez skupe graph analize
- **Automatski retry na nivou API-ja** - `withTransaction` Callback API automatski ponavlja transakciju kod `TransientTransactionError`, čime se privremeni konflikti transparentno prevazilaze

Tabela 7: Razrešavanje konflikata: klasični RDBMS nasuprot MongoDB

Aspekt	**Klasični RDBMS**	**MongoDB**
Detekcija	Analiza wait-for grafa	Timeout na lock zahtev
Razrešavanje	Žrtvovanje transakcije	`WriteConflict` + retry
Pristup	Pesimistički	Optimistički
Latencija detekcije	Zavisi od intervala provere	Definisana timeout-om
Rizično opterećenje	Duge konkurentne transakcije	Vruće tačke (hot spots)

```
// Promena timeout-a za pribavljanje lock-a (ms)
db.adminCommand({
  setParameter: 1,
  maxTransactionLockRequestTimeoutMillis: 10
});

// Praćenje write conflict-a na nivou servera
db.serverStatus().metrics.operation;
// "writeConflicts" - ukupan broj konflikata od pokretanja mongod-a

// Pregled operacija koje trenutno čekaju na lock
db.currentOp({ waitingForLock: true }).inprog.forEach(function(op) {
  print("OpId: " + op.opid +
        " | Trajanje (s): " + op.secs_running +
        " | Čeka lock: " + JSON.stringify(op.lockStats));
});
```

Listing 25: Podešavanje i praćenje konflikata lock-ova

Praktična posledica ovakvog pristupa je da MongoDB nikada ne prijavljuje eksplicitno *deadlock* stanje - sve pseudo-deadlock situacije manifestuju se kao `WriteConflict` greške.

8.7 MVCC Multiversion Concurrency Control

WiredTiger koristi MVCC (kontrolu konkurentnosti sa više verzija) za obezbeđivanje izolacije čitanja. Svaka operacija čitanja vidi konzistentan snimak podataka iz određenog trenutka, bez potrebe za zaključavanjem [15].

Ključne karakteristike MVCC implementacije u MongoDB-u:

- **Snapshot čitanje** - Svaka operacija čitanja radi sa snapshot-om podataka, što eliminiše blokiranja između čitača i pisača
- **In-memory upravljanje verzijama** - WiredTiger čuva više verzija dokumenta u memoriji dok postoje aktivne operacije koje ih referrenciraju
- **Checkpoint mehanizam** - WiredTiger periodično kreira checkpoint-e (svakih 60 sekundi) koji predstavljaju konzistentno stanje podataka na disku
- **Oslobađanje starih verzija** - Kada sve operacije koje referrenciraju staru verziju dokumenta završe, ta verzija se oslobađa iz memorije

8.8 Veza sa Dvofaznim zaključavanjem (2PL)

Protokol dvofaznog zaključavanja (*Two-Phase Locking, 2PL*) garantuje serijalizabilnost kroz dve faze: *growing* (sve operacije zaključavaju resurse) i *shrinking* (lock-ovi se oslobađaju). Striktna varijanta (Strict 2PL, S2PL) zahteva da se svi ekskluzivni lock-ovi drže do kraja transakcije, što eliminiše mogućnost kaskadnih poništavanja - transakcija ne može čitati nepotvrđene izmene druge transakcije jer ta transakcija još drži lock-ove.

Višedokumentne transakcije ponašaju se funkcionalno slično S2PL protokolu: lock-ovi na dokumentima drže se od prve modifikacije do commit-a ili abort-a. Ključna razlika je u implementaciji - MongoDB ne blokira čitaoce, već koristi kombinaciju intent lock-ova i MVCC-a:

- **Intent locks** na nivou baze i kolekcije koordinišu hijerarhijsko zaključavanje (analogna S2PL growing fazi)
- **Ekskluzivni lock na nivou dokumenta** (WiredTiger) drži se do commit-a (S2PL shrinking faza)
- **MVCC** omogućava čitaocima da vide prethodnu verziju dokumenta dok transakcija traje, bez blokiranja

```
session.startTransaction();

// Growing faza: svaka operacija pisanja zaključava dokument
// i drži lock do kraja transakcije
db.racuni.updateOne(
  { _id: "racun_A" }, { $inc: { stanje: -1000 } }, { session }
);  // <- lock na racun_A se drži

db.racuni.updateOne(
  { _id: "racun_B" }, { $inc: { stanje: 1000 } }, { session }
);  // <- lock na racun_B se drži

// Svi lock-ovi se drže do ovog trenutka
```

```
// Paralelni čitaoci vide staro stanje (MVCC) - ne blokiraju se
session.commitTransaction();
// Shrinking faza: svi lock-ovi se oslobađaju istovremeno
```

Listing 26: S2PL ponašanje MongoDB transakcija

Za razliku od klasičnog S2PL koji bi blokirao čitaoce koji čekaju na lock, MVCC pristup obezbeđuje visoku propusnost čitanja uz iste garancije sprečavanja kaskadnih poništavanja [15].

8.9 Upravljanje vrućim tačkama (hot spots)

Vruće tačke (hot spots) su dokumenti kojima istovremeno vise procesa pokušava da pristupi i gde ste stvaraju konflikti.

```

db.serverStatus().transactions;
// Visok odnos totalAborted / totalStarted ukazuje na hotspot
// Praćenje:
db.currentOp({ "waitingForLock": true }).inprog.length;
// Ako ovaj broj stalno raste - ukazuje na vruću tacku
```

Listing 27: Detekcija vrućih tačaka

Strategije za smanjenje vrucih tacaka u MongoDB kontekstu [3]:

- **Dekompozicija** - Umesto jednog dokumenta-brojača, koristiti niz dokumenata koji se periodično agregiraju. Svaka transakcija nasumično bira jedan od N dokumenata, smanjujući verovatnoću konflikta N puta
- **Smanjenje trajanja transakcija** - Kraće transakcije kraće drže lock-ove, smanjujući prozor za konflikt
- **Vremensko razdvajanje batch operacija** - Umesto konkurentnih transakcija nad istim dokumentima, koristiti red čekanja (queue pattern)
- **Atomični operatori van transakcija** - Kada je moguće, koristiti `$inc`, `$push` i sl. direktno bez transakcije

9 Preporuke za optimizaciju transakcija i konkurentnosti

9.1 Smanjenje trajanja transakcija

Kraće transakcije smanjuju verovatnoću write conflict-a i vreme zaključavanja dokumenata. Preporuke uključuju [3]:

- Izvršavati samo neophodne operacije unutar transakcije
- Izbegavati mrežnu komunikaciju ili korisnički unos unutar transakcije
- Pripremiti sve podatke pre pokretanja transakcije
- Koristiti indekse za brže pronalaženje dokumenata koji se modifikuju
- Razbiti velike transakcije na manje, nezavisne transakcije gde je to moguće

9.2 Izbor odgovarajućeg Read i Write Concern-a

Pravilan izbor nivoa konzistentnosti značajno utiče na performanse [10]:

- Za nekritične podatke (logovi, keš) - koristiti `w: 1` i `readConcern: "local"` za maksimalne performanse
- Za poslovne podatke - koristiti `w: "majority"` i `readConcern: "majority"` za balans između performansi i trajnosti
- Za kritične finansijske podatke - koristiti transakcije sa `readConcern: šnapshot"` i `writeConcern: {w: "majority", j: true}`

9.3 Strategije za smanjenje write conflict-a

- **Modelovanje podataka** - Minimizovati broj dokumenata koje transakcija modifikuje. Ugnježdavanje povezanih podataka u isti dokument eliminiše potrebu za transakcijom
- **Retry logika** - Implementirati automatske ponovne pokušaje koristeći Callback API (`withTransaction`), koji automatski ponavlja transakciju u slučaju prolaznih grešaka
- **Izbegavanje hotspot dokumenata** - Dokumenti koji se često ažuriraju od strane mnogo transakcija postaju uska grla. Razmotriti dekompoziciju takvih dokumenata
- **Indeksiranje** - Kreirati odgovarajuće indekse koji ubrzavaju pronalaženje dokumenata unutar transakcije, čime se smanjuje vreme držanja lock-ova

```
// Korišćenje withTransaction za automatski retry
function transferSredstava(saRacuna, naRacun, iznos) {
  const session = db.getMongo().startSession();

  try {
    session.withTransaction(function() {
      const racuni = session.getDatabase("banka").racuni;

      // Provera stanja pre transfera
      const izvor = racuni.findOne(
        { _id: saRacuna },
        { session }
      );

      if (izvor.stanje < iznos) {
        throw new Error("Nedovoljno sredstava");
      }

      // Atomični transfer
      racuni.updateOne(
        { _id: saRacuna },
        { $inc: { stanje: -iznos } },
        { session }
      );

      racuni.updateOne(
        { _id: naRacun },
        { $inc: { stanje: iznos } },
        { session }
      );
    }, {
      readConcern: { level: "snapshot" },
      writeConcern: { w: "majority" }
    });

    print("Transfer uspešan.");
  } finally {
    session.endSession();
  }
}
```

Listing 28: Optimizovana transakcija sa retry logikom

9.4 Praćenje performansi zaključavanja

Redovno praćenje metrika zaključavanja je neophodno za identifikaciju uskih grla [3]:

```
// Praćenje sporog upita (profiler)
db.setProfilingLevel(1, { slowms: 100 });

// Pregled sporih operacija
db.system.profile.find({
  millis: { $gt: 100 }
}).sort({ ts: -1 }).limit(10);

// Statistike transakcija na nivou servera
db.serverStatus().transactions;

```

```
// Rezultat uključuje:
// - totalStarted: ukupan broj započetih transakcija
// - totalCommitted: ukupan broj potvrđenih transakcija
// - totalAborted: ukupan broj poništenih transakcija

// Pregled operacija koje čekaju na lock
db.currentOp({
  "waitingForLock": true,
  "active": true
}).inprog.forEach(function(op) {
  print("OpId: " + op.opid +
        " | Tip: " + op.type +
        " | Čeka: " + op.waitingForLock +
        " | Trajanje (s): " + op.secs_running);
});
```

Listing 29: Praćenje metrika zaključavanja

10 Zaključak

Obrada transakcija u MongoDB-u je prošla značajnu evoluciju - od baze podataka koja se oslanjala isključivo na jednodokumentnu atomičnost do sistema sa punom podrškom za višedokumentne ACID transakcije. Dokumentni model podataka je i dalje ključna prednost MongoDB-a, jer omogućava modelovanje koje u mnogim slučajevima eliminiše potrebu za ugnježdavanjem i denormalizaciju podataka.

Planovi izvršavanja pružaju detaljan uvid u obradu upita. Razumevanje faza plana (COLLSCAN, IXSCAN, FETCH, SORT) i ključnih metrika omogućava identifikaciju neefikasnih upita i projektovanje odgovarajućih indeksa po ESR pravilu.

Mehanizmi Read Concern i Write Concern pružaju fleksibilnu kontrolu nad konzistencijom i trajnošću.

U domenu izolacije, MongoDB-ova snapshot izolacija sprečava dirty read, lost update, non-repeatable read i phantom anomalije. Optimistička kontrola konkurentnosti kroz write conflict detekciju obezbeđuje visoku propusnost, ali može dovesti do thrashing-a kod vrućih tačaka - što se rešava dekompozicijom dokumenata i smanjivanjem trajanja transakcija.

WiredTiger storage engine, sa zaključavanjem na nivou dokumenta i MVCC mehanizmom, postiže ponašanje funkcionalno ekvivalentno Strict 2PL protokolu: lock-ovi na dokumentima drže se do commit-a, sprečavajući kaskadna poništavanja, dok MVCC obezbeđuje da čitaoci ne blokiraju pisce.

Za optimalne performanse u konkurentnom okruženju, ključno je kombinovati sve navedene mehanizme: smanjiti trajanje transakcija, pravilno modelovati podatke, izabrati odgovarajuće nivoe Read i Write Concern-a, pratiti metrike zaključavanja i konfigurisati adekvatnu replikacionu strategiju.

11 Literatura

[1] Shannon Bradshaw, Eoin Brazil i Kristina Chodorow. *MongoDB: The Definitive Guide*. 3rd. Sebastopol, CA: O'Reilly Media, 2019. ISBN: 978-1491954461.

[2] MongoDB, Inc. *Transactions — MongoDB Manual*. MongoDB. 2025. URL: https://www.mongodb.com/docs/manual/core/transactions/ (posećeno 15. 12. 2025.).

[3] Guy Harrison i Michael Harrison. *MongoDB Performance Tuning: Optimizing MongoDB Databases and their Applications*. New York, NY: Apress, 2021. ISBN: 978-1484268780.

[4] Amit Phaltankar i dr. *MongoDB Fundamentals: A hands-on guide to using MongoDB and Atlas in the real world*. Birmingham, UK: Packt Publishing, 2020. ISBN: 978-1839210648.

[5] MongoDB, Inc. *Transactions and Sharded Clusters — MongoDB Manual*. MongoDB. 2025. URL: https://www.mongodb.com/docs/manual/core/transactions-sharded-clusters/ (posećeno 15. 12. 2025.).

[6] Andy Schwerin i dr. „Implementation of Cluster-wide Logical Clock and Causal Consistency in MongoDB". U: *Proceedings of the ACM SIGMOD International Conference on Management of Data* (2019.), str. 636–650.

[7] MongoDB, Inc. *Explain Results — MongoDB Manual*. MongoDB. 2025. URL: https://www.mongodb.com/docs/manual/reference/explain-results/ (posećeno 15. 12. 2025.).

[8] MongoDB, Inc. *Aggregation Pipeline Optimization — MongoDB Manual*. MongoDB. 2025. URL: https://www.mongodb.com/docs/manual/core/aggregation-pipeline-optimization/ (posećeno 15. 12. 2025.).

[9] MongoDB, Inc. *Indexes — MongoDB Manual*. MongoDB. 2025. URL: https://www.mongodb.com/docs/manual/indexes/ (posećeno 15. 12. 2025.).

[10] MongoDB, Inc. *Write Concern — MongoDB Manual*. MongoDB. 2025. URL: https://www.mongodb.com/docs/manual/reference/write-concern/ (posećeno 15. 12. 2025.).

[11] MongoDB, Inc. *Read Concern — MongoDB Manual*. MongoDB. 2025. URL: https://www.mongodb.com/docs/manual/reference/read-concern/ (posećeno 15. 12. 2025.).

[12] MongoDB, Inc. *Read Isolation, Consistency, and Recency — MongoDB Manual*. MongoDB. 2025. URL: https://www.mongodb.com/docs/manual/core/read-isolation-consistency-recency/ (posećeno 15. 12. 2025.).

[13] Raghu Ramakrishnan i Johannes Gehrke. *Database Management Systems*. 3rd. New York, NY: McGraw-Hill, 2003. ISBN: 978-0072465631.

[14] MongoDB, Inc. *WiredTiger Storage Engine — MongoDB Manual*. MongoDB. 2025. URL: https://www.mongodb.com/docs/manual/core/wiredtiger/ (posećeno 15. 12. 2025.).

[15] MongoDB, Inc. *FAQ: Concurrency — MongoDB Manual*. MongoDB. 2025. URL: https://www.mongodb.com/docs/manual/faq/concurrency/ (posećeno 15. 12. 2025.).